Pack 18 (Desktop) — Host→API Audit Forwarding + Session Event Timeline + Viewer Reconnect Policy Re-check (real code)

RemoteDesk · Pack 18 (Desktop) — Host→API Audit Forwarding + Session Event Timeline + Viewer Reconnect Policy Re-check (real code)

> Codex / Claude Code handoff. Direct continuation of Desktop Packs 15 → 17. This slice closes the audit loop and hardens reconnects: (1) batched, fail-silent forwarding of the Pack 17 session audit events to the API audit log, (2) a session-event timeline UI so host + viewer can see what happened, and (3) a viewer reconnect flow that RE-FETCHES and RE-EVALUATES policy before any interactive feature comes back. Build order: shared reconnect contract → audit forwarder → timeline store → reconnect controller → UI → wiring → tests.
> Scope: make audit durable (without blocking sessions), visible (timeline), and make reconnects fail-closed (never resume input on stale permissions). **Fail-closed everywhere. Remote input still off by default. Host stays source of truth. No native executor. No secrets/payloads in logs or forwarded audit.**
> Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

```
CREATED [NEW]
  packages/shared/src/permissions/reconnect.ts
  apps/desktop/src/renderer/src/services/auditForwarder.ts
  apps/desktop/src/renderer/src/services/sessionTimelineStore.ts
  apps/desktop/src/renderer/src/services/reconnectPolicyController.ts
  apps/desktop/src/renderer/src/components/SessionTimelinePanel.tsx
  apps/desktop/src/renderer/src/components/ReconnectBanner.tsx
  tests/permissions/reconnect.test.ts
  tests/permissions/auditForwarder.test.ts
  tests/permissions/sessionTimelineStore.test.ts

MODIFIED [MODIFY]
  packages/shared/src/permissions/index.ts                        (export reconnect)
  apps/desktop/src/renderer/src/services/sessionAuditHooks.ts     (fan-out to forwarder + timeline)
  apps/desktop/src/renderer/src/services/socket.ts                (reconnect -> policy re-check)
  apps/desktop/src/renderer/src/components/RemoteSessionView.tsx   (timeline panel + reconnect banner)

NOT TOUCHED (intentionally)
  apps/desktop/src/main/index.ts          (no native executor)
  apps/desktop/src/preload/index.ts       (no new bridge)
  apps/desktop/src/renderer/src/services/webrtc.ts          (Pack 15/17 gates unchanged)
  apps/desktop/src/renderer/src/services/permissionState.ts (Pack 15/16 unchanged)
  apps/api/**  apps/web/**                (reuses Pack 14 POST /api/audit/logs)
```

Part A — Shared: reconnect contract

FILE: packages/shared/src/permissions/reconnect.ts

```
import {
  evaluateSessionStartPolicy,
  type DeviceAccessPolicySnapshot,
  type PolicyDecision,
} from './policyEvaluator';

/**
 * Reconnect lifecycle for a desktop session. On any transport drop the session
 * must NOT silently resume interactive features — it re-fetches policy and
 * re-evaluates before control can come back. Fail closed: while reconnecting,
 * interactive features are treated as denied.
 */
export type ReconnectPhase =
  | 'connected'
  | 'reconnecting'
  | 'rechecking_policy'
  | 'recovered'
  | 'failed';

export interface ReconnectState {
  phase: ReconnectPhase;
  /** How many reconnect attempts so far this drop. */
  attempt: number;
  /** ISO of the last phase change. */
  changedAt: string;
  /** Set when phase === 'failed'. */
  failureReason: string | null;
}

export function initialReconnectState(now: Date = new Date()): ReconnectState {
  return { phase: 'connected', attempt: 0, changedAt: now.toISOString(), failureReason: null };
}

/** Are interactive features allowed to be active in this phase? Only when fully connected. */
export function interactiveAllowedInPhase(phase: ReconnectPhase): boolean {
  return phase === 'connected' || phase === 'recovered';
}

/**
 * Decide whether control may resume after a reconnect, given the freshly
 * re-fetched policy. Reuses the same start-gate so a device that became blocked
 * while offline cannot resume. Fail closed on a missing policy.
 */
export function evaluateReconnectResume(
  policy: DeviceAccessPolicySnapshot | null,
): PolicyDecision {
  return evaluateSessionStartPolicy(policy);
}

/** Exponential backoff (ms) for attempt N, capped. Pure + deterministic. */
export function backoffMs(attempt: number, baseMs = 500, capMs = 10_000): number {
  const ms = baseMs * 2 ** Math.max(0, attempt - 1);
```

```
  return Math.min(capMs, ms);
}
```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```
export * from './devicePolicy';
export * from './policyEvaluator';
export * from './sessionApproval';
export * from './inputAction';
export * from './sessionAuditEvents';
export * from './reconnect'; // ADD
```

Part B — Desktop: audit forwarder

FILE: apps/desktop/src/renderer/src/services/auditForwarder.ts

```
import { isAuditDetailSafe, type SessionAuditEvent } from '@remotedesk/shared';

/**
 * Batches Pack 17 session audit events and forwards them to the API audit log
 * (Pack 14: POST /api/audit/logs). FAILS SILENTLY and never blocks the session:
 * a failed flush re-queues the batch (bounded) and retries on the next tick.
 * Drops any event with sensitive-looking detail before it can leave the device.
 */
export type AuditTransport = (events: SessionAuditEvent[]) => Promise<void>;

export interface AuditForwarderOptions {
  flushIntervalMs?: number;
  maxBatch?: number;
  maxQueue?: number;
}

export class AuditForwarder {
  private queue: SessionAuditEvent[] = [];
  private timer: ReturnType<typeof setInterval> | null = null;
  private readonly flushIntervalMs: number;
  private readonly maxBatch: number;
  private readonly maxQueue: number;

  constructor(private readonly transport: AuditTransport, opts: AuditForwarderOptions = {}) {
    this.flushIntervalMs = opts.flushIntervalMs ?? 5000;
    this.maxBatch = opts.maxBatch ?? 50;
    this.maxQueue = opts.maxQueue ?? 1000;
  }

  enqueue(event: SessionAuditEvent): void {
    if (!isAuditDetailSafe(event.detail)) return; // never forward unsafe detail
    this.queue.push(event);
    if (this.queue.length > this.maxQueue) this.queue.shift(); // drop oldest
  }

  start(): void {
    if (this.timer) return;
    this.timer = setInterval(() => void this.flush(), this.flushIntervalMs);
  }
```

```
  stop(): void {
    if (this.timer) clearInterval(this.timer);
    this.timer = null;
  }

  /** Flush one batch. Re-queues on failure. Exposed for tests + manual flush. */
  async flush(): Promise<void> {
    if (this.queue.length === 0) return;
    const batch = this.queue.splice(0, this.maxBatch);
    try {
      await this.transport(batch);
    } catch {
      // Re-queue at the front (bounded) and try again next tick. Fail silent.
      this.queue = [...batch, ...this.queue].slice(0, this.maxQueue);
    }
  }

  pending(): number {
    return this.queue.length;
  }
}

/**
 * Default transport over the Pack 14 audit endpoint. Maps each session event to
 * an audit log row. Throws on non-2xx so the forwarder re-queues.
 */
export function httpAuditTransport(baseUrl: string, authToken: string): AuditTransport {
  return async (events) => {
    for (const e of events) {
      const res = await fetch(`${baseUrl}/api/audit/logs`, {
        method: 'POST',
        headers: { 'content-type': 'application/json', authorization: `Bearer ${authToken}` },
        body: JSON.stringify({
          action: mapToAuditAction(e.type),
          metadata: stringifyDetail(e.detail),
        }),
      });
      if (!res.ok) throw new Error(`audit forward failed: ${res.status}`);
    }
  };
}

/** Map session audit type -> Pack 14 AuditAction. Unknown -> input.permission.changed. */
function mapToAuditAction(type: SessionAuditEvent['type']): string {
  switch (type) {
    case 'control.granted':
    case 'control.revoked':
    case 'emergency_stop.engaged':
    case 'emergency_stop.released':
      return 'input.permission.changed';
    case 'session.expired':
      return 'session.end';
    default:
      return 'input.permission.changed';
  }
}
```

```
/** Coerce detail values to strings (Pack 14 metadata is Record<string,string>). */
function stringifyDetail(detail: Record<string, string | number | boolean>): Record<string, string> {
  const out: Record<string, string> = {};
  for (const [k, v] of Object.entries(detail)) out[k] = String(v);
  return out;
}
```

Part C — Desktop: session timeline store

FILE: apps/desktop/src/renderer/src/services/sessionTimelineStore.ts

```
import type { SessionAuditEvent } from '@remotedesk/shared';

/**
 * In-memory, bounded timeline of session audit events for display. Read-only to
 * the UI; the audit hooks push into it. Newest-last ordering. Holds counts so a
 * compact summary can render without scanning the whole list.
 */
export class SessionTimelineStore {
  private events: SessionAuditEvent[] = [];
  private counts = new Map<string, number>();
  private listeners = new Set<() => void>();

  constructor(private readonly max = 300) {}

  push(event: SessionAuditEvent): void {
    this.events.push(event);
    if (this.events.length > this.max) this.events.shift();
    this.counts.set(event.type, (this.counts.get(event.type) ?? 0) + 1);
    this.emit();
  }

  all(): readonly SessionAuditEvent[] {
    return this.events;
  }

  recent(limit = 50): SessionAuditEvent[] {
    return this.events.slice(-limit);
  }

  countOf(type: string): number {
    return this.counts.get(type) ?? 0;
  }

  subscribe(fn: () => void): () => void {
    this.listeners.add(fn);
    return () => this.listeners.delete(fn);
  }

  private emit(): void {
    for (const fn of this.listeners) fn();
  }
}
```

FILE: apps/desktop/src/renderer/src/services/sessionAuditHooks.ts [MODIFY]

```
// ...existing Pack 17 imports...
import type { AuditForwarder } from './auditForwarder';
import type { SessionTimelineStore } from './sessionTimelineStore';

// ---- MODIFY: let SessionAuditHooks fan out to a forwarder + timeline ----
// Extend the constructor (keep the optional sink from Pack 17):
//
// constructor(
//   private readonly sink?: AuditSink,
//   private readonly maxBuffer = 500,
//   private readonly forwarder?: AuditForwarder,   // ADD
//   private readonly timeline?: SessionTimelineStore, // ADD
// ) {}
//
// In record(), after the existing safe-detail check + buffer push, also fan out:
//   this.timeline?.push(event);
//   this.forwarder?.enqueue(event);
//
// Everything still fails silently; forwarder + timeline are optional.
```

Part D — Desktop: reconnect policy controller

FILE: apps/desktop/src/renderer/src/services/reconnectPolicyController.ts

```
import {
  initialReconnectState,
  interactiveAllowedInPhase,
  evaluateReconnectResume,
  makeAuditEvent,
  type DeviceAccessPolicySnapshot,
  type ReconnectState,
  type SessionAuditEvent,
} from '@remotedesk/shared';

/**
 * Drives reconnect with a mandatory policy re-check. On drop it forces the host
 * permission state to revoke interactive features, and only after a fresh policy
 * fetch + re-evaluation does it allow control to resume. Fail closed: during
 * reconnecting/rechecking, interactive features stay off; a blocked/missing
 * policy keeps them off and ends in 'failed'.
 */
export interface ReconnectDeps {
  /** Re-fetch the authoritative policy (returns null on failure -> fail closed). */
  refetchPolicy: () => Promise<DeviceAccessPolicySnapshot | null>;
  /** Push the re-fetched policy into the host permission state. */
  applyPolicy: (policy: DeviceAccessPolicySnapshot | null) => void;
  /** Force interactive features off immediately (e.g. host setHostAccepted(false)). */
  revokeInteractive: () => void;
  /** Emit a session audit event. */
  onAudit?: (e: SessionAuditEvent) => void;
}
```

```
export class ReconnectPolicyController {
  private state: ReconnectState = initialReconnectState();
  private listeners = new Set<(s: ReconnectState) => void>();

  constructor(private readonly deps: ReconnectDeps) {}

  current(): ReconnectState {
    return this.state;
  }

  interactiveAllowed(): boolean {
    return interactiveAllowedInPhase(this.state.phase);
  }

  /** Transport dropped: revoke interactive immediately + enter reconnecting. */
  onDrop(): void {
    this.deps.revokeInteractive(); // fail closed NOW
    this.deps.onAudit?.(makeAuditEvent('control.revoked', { reason: 'transport_drop' }));
    this.transition('reconnecting', { attempt: this.state.attempt + 1 });
  }

  /**
   * Transport re-established: re-check policy BEFORE allowing control back.
   * Returns true if control may resume, false if it stays denied/failed.
   */
  async onTransportRestored(): Promise<boolean> {
    this.transition('rechecking_policy');
    const policy = await this.deps.refetchPolicy();
    this.deps.applyPolicy(policy); // push fresh (or null) policy in

    const decision = evaluateReconnectResume(policy);
    if (!decision.allowed) {
      this.transition('failed', { failureReason: decision.reason });
      return false;
    }
    this.transition('recovered');
    return true;
  }

  /** Give up after exhausting attempts. */
  fail(reason: string): void {
    this.deps.revokeInteractive();
    this.transition('failed', { failureReason: reason });
  }

  subscribe(fn: (s: ReconnectState) => void): () => void {
    this.listeners.add(fn);
    return () => this.listeners.delete(fn);
  }

  private transition(phase: ReconnectState['phase'], over: Partial<ReconnectState> = {}): void {
    this.state = {
      ...this.state,
      phase,
      changedAt: new Date().toISOString(),
      failureReason: over.failureReason ?? (phase === 'failed' ? this.state.failureReason : null),
      ...over,
```

```
    };
    for (const fn of this.listeners) fn(this.state);
  }
}
```

Part E — Desktop: socket wiring

FILE: apps/desktop/src/renderer/src/services/socket.ts [MODIFY]

```
// ...existing imports...
import { ReconnectPolicyController } from './reconnectPolicyController';

// ---- ADD: construct once with deps wired to policy client + permission state ----
// export const reconnectController = new ReconnectPolicyController({
//   refetchPolicy: () => devicePolicyClient.fetch(deviceId).then((r) => r.policy),
//   applyPolicy: (p) => hostPermissionState.setPolicy(p),
//   revokeInteractive: () => hostPermissionState.setHostAccepted(false),
//   onAudit: (e) => sessionAuditHooks.record(e),
// });

// ---- MODIFY: hook into existing socket lifecycle events ----
// socket.on('disconnect', () => {
//   reconnectController.onDrop();   // revokes interactive + enters reconnecting
// });
// socket.on('reconnect', async () => {
//   const resumed = await reconnectController.onTransportRestored(); // re-checks policy
//   // resumed === false => control stays off; UI shows the reconnect banner state
// });
//
// NOTE: signaling/connection logic itself is unchanged — we only react to the
// existing disconnect/reconnect events. Screen view recovery stays as-is.
```

Part F — UI

FILE: apps/desktop/src/renderer/src/components/SessionTimelinePanel.tsx

```
import React, { useEffect, useState } from 'react';
import type { SessionAuditEvent } from '@remotedesk/shared';
import type { SessionTimelineStore } from '../services/sessionTimelineStore';

const LABELS: Record<string, string> = {
  'control.granted': 'Control granted',
  'control.revoked': 'Control revoked',
  'emergency_stop.engaged': 'Emergency stop engaged',
  'emergency_stop.released': 'Emergency stop released',
  'input.blocked': 'Input blocked',
  'input.rate_limited': 'Input rate-limited',
  'session.expired': 'Session expired',
};

/**
 * Read-only session event timeline. Renders non-sensitive audit events (type +
 * time + small detail). Shared component; show on host and viewer.
 */
```

```
export function SessionTimelinePanel({ store }: { store: SessionTimelineStore }) {
  const [events, setEvents] = useState<SessionAuditEvent[]>(store.recent());

  useEffect(() => store.subscribe(() => setEvents(store.recent())), [store]);

  if (events.length === 0) {
    return <div className="timeline-panel timeline-panel--empty">No session events yet.</div>;
  }
  return (
    <ul className="timeline-panel">
      {events.map((e, i) => (
        <li key={`${e.at}-${i}`} className={`timeline-item timeline-item--${e.type.split('.')[0]}`}>
          <span className="timeline-item__time">{new Date(e.at).toLocaleTimeString()}</span>
          <span className="timeline-item__label">{LABELS[e.type] ?? e.type}</span>
          {'reason' in e.detail && <span className="timeline-item__detail">({String(e.detail.reason)})</span>}
        </li>
      ))}
    </ul>
  );
}
```

FILE: apps/desktop/src/renderer/src/components/ReconnectBanner.tsx

```
import React from 'react';
import type { ReconnectState } from '@remotedesk/shared';

/**
 * Shows reconnect status. While reconnecting/rechecking, makes it explicit that
 * interactive control is paused (fail closed). On 'failed', surfaces the reason.
 */
export function ReconnectBanner({ state }: { state: ReconnectState }) {
  if (state.phase === 'connected' || state.phase === 'recovered') return null;

  const text =
    state.phase === 'reconnecting'
      ? `Reconnecting (attempt ${state.attempt})… remote control paused.`
      : state.phase === 'rechecking_policy'
        ? 'Reconnected. Re-checking device policy before resuming control…'
        : `Could not safely resume control${state.failureReason ? ` (${state.failureReason})` : ''}. View-only.`;

  const tone = state.phase === 'failed' ? 'danger' : 'warn';
  return (
    <div className={`reconnect-banner reconnect-banner--${tone}`} role="status">
      {text}
    </div>
  );
}
```

FILE: apps/desktop/src/renderer/src/components/RemoteSessionView.tsx [MODIFY]

```
// ...existing imports + Pack 15/16/17 additions...
import { SessionTimelinePanel } from './SessionTimelinePanel';
import { ReconnectBanner } from './ReconnectBanner';
import { reconnectController } from '../services/socket';
import type { ReconnectState } from '@remotedesk/shared';
```

```
// ---- ADD: track reconnect state ----
// const [reconnect, setReconnect] = useState<ReconnectState>(reconnectController.current());
// useEffect(() => reconnectController.subscribe(setReconnect), []);

// ---- ADD: while not interactive-allowed, force viewer read-only regardless of
// the last permission snapshot (defense in depth on top of the gates) ----
// const interactiveAllowed = reconnectController.interactiveAllowed();
// const readOnly = !interactiveAllowed || !remoteInputGate.inputAllowed();

// ---- ADD to render: reconnect banner up top, timeline panel in a side/footer area ----
// <ReconnectBanner state={reconnect} />
// ... existing PolicyStatusBanner / EmergencyStopButton / SessionApprovalPrompt / SessionExpiryBadge /
InputActivityMeter ...
// <SessionTimelinePanel store={sessionTimelineStore} />
//
// sessionTimelineStore is the SessionTimelineStore constructed at session start and
// passed into SessionAuditHooks (Part C wiring).
```

Part G — Tests

FILE: tests/permissions/reconnect.test.ts

```
import { describe, it, expect } from 'vitest';
import {
  initialReconnectState,
  interactiveAllowedInPhase,
  evaluateReconnectResume,
  backoffMs,
  type DeviceAccessPolicySnapshot,
} from '@remotedesk/shared';

function policy(over: Partial<DeviceAccessPolicySnapshot> = {}): DeviceAccessPolicySnapshot {
  return {
    deviceId: 'dev-1', trustStatus: 'trusted', unattendedAccessEnabled: false,
    remoteInputEnabled: true, clipboardSyncEnabled: true, fileTransferEnabled: true,
    requiresSessionApproval: true, maxSessionMinutes: null, updatedAt: new Date().toISOString(),
    ...over,
  };
}

describe('reconnect', () => {
  it('starts connected', () => {
    expect(initialReconnectState().phase).toBe('connected');
  });

  it('only allows interactive features when connected/recovered', () => {
    expect(interactiveAllowedInPhase('connected')).toBe(true);
    expect(interactiveAllowedInPhase('recovered')).toBe(true);
    expect(interactiveAllowedInPhase('reconnecting')).toBe(false);
    expect(interactiveAllowedInPhase('rechecking_policy')).toBe(false);
    expect(interactiveAllowedInPhase('failed')).toBe(false);
  });

  it('blocks resume on a missing policy (fail closed)', () => {
    expect(evaluateReconnectResume(null).reason).toBe('missing_policy');
```

```
  });

  it('blocks resume if the device became blocked while offline', () => {
    expect(evaluateReconnectResume(policy({ trustStatus: 'blocked' })).reason).toBe('device_blocked');
  });

  it('allows resume for a healthy policy', () => {
    expect(evaluateReconnectResume(policy()).allowed).toBe(true);
  });

  it('computes capped exponential backoff', () => {
    expect(backoffMs(1, 500, 10_000)).toBe(500);
    expect(backoffMs(2, 500, 10_000)).toBe(1000);
    expect(backoffMs(3, 500, 10_000)).toBe(2000);
    expect(backoffMs(20, 500, 10_000)).toBe(10_000); // capped
  });
});
```

FILE: tests/permissions/auditForwarder.test.ts

```
import { describe, it, expect, vi } from 'vitest';
import { AuditForwarder } from '../../apps/desktop/src/renderer/src/services/auditForwarder';
import { makeAuditEvent } from '@remotedesk/shared';

describe('AuditForwarder', () => {
  it('forwards queued events on flush', async () => {
    const transport = vi.fn().mockResolvedValue(undefined);
    const fwd = new AuditForwarder(transport, { maxBatch: 10 });
    fwd.enqueue(makeAuditEvent('control.granted', { by: 'host' }));
    fwd.enqueue(makeAuditEvent('control.revoked', { reason: 'transport_drop' }));
    await fwd.flush();
    expect(transport).toHaveBeenCalledTimes(1);
    expect(transport.mock.calls[0][0]).toHaveLength(2);
    expect(fwd.pending()).toBe(0);
  });

  it('re-queues on transport failure (fail silent)', async () => {
    const transport = vi.fn().mockRejectedValue(new Error('offline'));
    const fwd = new AuditForwarder(transport);
    fwd.enqueue(makeAuditEvent('emergency_stop.engaged'));
    await fwd.flush(); // throws internally, swallowed
    expect(fwd.pending()).toBe(1); // re-queued, not lost
  });

  it('drops events with unsafe detail before forwarding', () => {
    const fwd = new AuditForwarder(vi.fn());
    // @ts-expect-error intentionally unsafe detail key
    fwd.enqueue(makeAuditEvent('input.blocked', { token: 'secret' }));
    expect(fwd.pending()).toBe(0);
  });

  it('bounds the queue, dropping oldest', () => {
    const fwd = new AuditForwarder(vi.fn(), { maxQueue: 2 });
    fwd.enqueue(makeAuditEvent('input.blocked', { seq: 1 }));
    fwd.enqueue(makeAuditEvent('input.blocked', { seq: 2 }));
    fwd.enqueue(makeAuditEvent('input.blocked', { seq: 3 }));
```

```
    expect(fwd.pending()).toBe(2);
  });
});
```

FILE: tests/permissions/sessionTimelineStore.test.ts

```
import { describe, it, expect, vi } from 'vitest';
import { SessionTimelineStore } from '../../apps/desktop/src/renderer/src/services/sessionTimelineStore';
import { makeAuditEvent } from '@remotedesk/shared';

describe('SessionTimelineStore', () => {
  it('stores events newest-last and counts by type', () => {
    const store = new SessionTimelineStore();
    store.push(makeAuditEvent('control.granted'));
    store.push(makeAuditEvent('input.blocked', { seq: 1 }));
    store.push(makeAuditEvent('input.blocked', { seq: 2 }));
    expect(store.all()).toHaveLength(3);
    expect(store.countOf('input.blocked')).toBe(2);
    expect(store.recent(1)[0].type).toBe('input.blocked');
  });

  it('bounds the buffer', () => {
    const store = new SessionTimelineStore(2);
    store.push(makeAuditEvent('control.granted'));
    store.push(makeAuditEvent('control.revoked'));
    store.push(makeAuditEvent('emergency_stop.engaged'));
    expect(store.all()).toHaveLength(2);
    expect(store.all()[0].type).toBe('control.revoked'); // oldest dropped
  });

  it('notifies subscribers on push', () => {
    const store = new SessionTimelineStore();
    const fn = vi.fn();
    store.subscribe(fn);
    store.push(makeAuditEvent('session.expired'));
    expect(fn).toHaveBeenCalledTimes(1);
  });
});
```

Part H — Reports

FILE: generated-pack18-desktop-audit-reconnect-merge-summary.md

```
# Pack 18 (Desktop) — Audit Forwarding + Timeline + Reconnect Re-check · Merge Summary

## What this slice does
Continues Desktop Packs 15 → 17. Closes the audit loop and hardens reconnects:
1. AuditForwarder batches Pack 17 session events and POSTs them to the Pack 14
   audit log, failing silently + re-queueing on error.
2. SessionTimelineStore + SessionTimelinePanel give a live, read-only event
   timeline (non-sensitive).
3. ReconnectPolicyController revokes interactive control on a drop and RE-FETCHES +
   RE-EVALUATES policy before allowing control to resume — fail closed throughout.

## Files created (9)
```

```
- packages/shared/src/permissions/reconnect.ts
- apps/desktop/src/renderer/src/services/auditForwarder.ts
- apps/desktop/src/renderer/src/services/sessionTimelineStore.ts
- apps/desktop/src/renderer/src/services/reconnectPolicyController.ts
- apps/desktop/src/renderer/src/components/SessionTimelinePanel.tsx
- apps/desktop/src/renderer/src/components/ReconnectBanner.tsx
- tests/permissions/reconnect.test.ts
- tests/permissions/auditForwarder.test.ts
- tests/permissions/sessionTimelineStore.test.ts

## Files modified (4)
- packages/shared/src/permissions/index.ts — export reconnect.
- apps/desktop/src/renderer/src/services/sessionAuditHooks.ts — fan out to forwarder + timeline.
- apps/desktop/src/renderer/src/services/socket.ts — disconnect/reconnect -> controller.
- apps/desktop/src/renderer/src/components/RemoteSessionView.tsx — reconnect banner + timeline panel + read-only during
reconnect.

## Files intentionally NOT touched
- main/index.ts, preload/index.ts — no native executor / no bridge.
- webrtc.ts, permissionState.ts — Pack 15/16/17 gates unchanged.
- apps/api/**, apps/web/** — reuses Pack 14 POST /api/audit/logs.

## Remaining manual wiring risks
1. Construct AuditForwarder + SessionTimelineStore at session start; pass both into SessionAuditHooks; start()/stop() the
forwarder with the session.
2. Construct ReconnectPolicyController in socket.ts with refetchPolicy/applyPolicy/revokeInteractive wired to the real
policy client + host permission state.
3. Hook controller.onDrop()/onTransportRestored() into the existing socket disconnect/reconnect events.
4. In RemoteSessionView, force read-only when reconnectController.interactiveAllowed() is false.

## Verify
- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: kill the socket mid-session -> control revokes instantly + banner shows reconnecting; on reconnect, if the
device was blocked server-side, control stays off (failed); audit rows land in /api/audit/logs after a flush.
```

FILE: generated-pack18-desktop-audit-reconnect-code-review.md

```
# Pack 18 (Desktop) — Code Review Notes

## Strengths
- Reconnect is fail-closed by construction: drop revokes interactive immediately; resume requires a fresh policy fetch +
re-eval; missing/blocked policy ends in 'failed'.
- Reuses the SAME start-gate (evaluateSessionStartPolicy) so a device blocked while offline cannot resume.
- Audit forwarder never blocks the session: bounded queue, re-queue on failure, drops unsafe detail before send.
- Timeline is read-only + non-sensitive (type/time/reason only).
- Pure reconnect math (phases, backoff, resume gate) fully unit-tested; forwarder + store tested in isolation.

## Reviewer checklist
- [ ] SessionAuditHooks constructed with forwarder + timeline; forwarder.start()/stop() tied to session lifecycle.
- [ ] ReconnectPolicyController deps wired to the real devicePolicyClient + hostPermissionState.
- [ ] Socket disconnect/reconnect events call onDrop/onTransportRestored.
- [ ] RemoteSessionView forces read-only while interactiveAllowed() is false.
- [ ] No payloads/keys forwarded: mapToAuditAction + stringifyDetail only carry safe detail.

## Safety
- Interactive control cannot silently resume after a reconnect.
```

```
- Audit forwarding is fail-silent + non-sensitive; secrets/payloads never leave the device.
- Native OS execution remains disabled/no-op (untouched).
```

FILE: generated-pack18-desktop-audit-reconnect-manifest.json

```
{
  "pack": "pack18-desktop-audit-reconnect",
  "title": "Host->API Audit Forwarding + Session Event Timeline + Viewer Reconnect Policy Re-check",
  "actualFileCount": 13,
  "filesCreated": [
    "packages/shared/src/permissions/reconnect.ts",
    "apps/desktop/src/renderer/src/services/auditForwarder.ts",
    "apps/desktop/src/renderer/src/services/sessionTimelineStore.ts",
    "apps/desktop/src/renderer/src/services/reconnectPolicyController.ts",
    "apps/desktop/src/renderer/src/components/SessionTimelinePanel.tsx",
    "apps/desktop/src/renderer/src/components/ReconnectBanner.tsx",
    "tests/permissions/reconnect.test.ts",
    "tests/permissions/auditForwarder.test.ts",
    "tests/permissions/sessionTimelineStore.test.ts"
  ],
  "filesModified": [
    "packages/shared/src/permissions/index.ts",
    "apps/desktop/src/renderer/src/services/sessionAuditHooks.ts",
    "apps/desktop/src/renderer/src/services/socket.ts",
    "apps/desktop/src/renderer/src/components/RemoteSessionView.tsx"
  ],
  "filesNotTouched": [
    "apps/desktop/src/main/index.ts",
    "apps/desktop/src/preload/index.ts",
    "apps/desktop/src/renderer/src/services/webrtc.ts",
    "apps/desktop/src/renderer/src/services/permissionState.ts",
    "apps/api/**",
    "apps/web/**"
  ],
  "dependenciesRequired": {
    "new": [],
    "existing": ["@remotedesk/shared", "react", "vitest"],
    "assumedFromProject": [
      "Desktop Pack 15 HostPermissionState + devicePolicyClient",
      "Desktop Pack 17 SessionAuditHooks + makeAuditEvent",
      "Pack 14 POST /api/audit/logs endpoint",
      "existing socket with disconnect/reconnect events"
    ]
  },
  "knownLimitations": [
    "Audit forwarding maps session events onto Pack 14 AuditActions (input.permission.changed / session.end); add
dedicated actions if finer granularity is needed.",
    "Timeline is in-memory + session-local (not persisted client-side).",
    "Reconnect controller reacts to existing socket events; it does not change signaling/connection logic.",
    "Native OS input execution remains disabled/no-op."
  ],
  "remainingManualWiringRisks": [
    "Wire forwarder + timeline into SessionAuditHooks and start/stop the forwarder with the session.",
    "Construct ReconnectPolicyController with real deps and hook socket disconnect/reconnect.",
    "Force RemoteSessionView read-only while interactiveAllowed() is false."
  ],
```

```
    "estimatedCompletionAfterCarefulMerge": "66-72%"
  }
```

Desktop Pack 18 done: 13 files (9 created, 4 modified). Closes the audit loop and hardens reconnects. The Pack 17 audit events now batch-forward to the Pack 14 audit log (fail-silent, re-queue on error, drops unsafe detail), a read-only session timeline shows what happened, and reconnects can no longer silently resume control.

The clean bit: the reconnect controller revokes interactive features the instant the transport drops, then refuses to bring control back until it re-fetches the policy and runs it through the SAME start-gate, so a device that got blocked while you were offline stays blocked on reconnect. The screen view recovers normally; only control is gated. Audit forwarding never blocks the session and never carries payloads or keys off the device.

Next slice = clipboard/file-transfer reconnect resync + transfer resumption guards, or a viewer-side trust-prompt + first-connect device fingerprint. Bolo.